WORK_DISK

PHASE ONE — OFFICIAL ROOT DOCUMENT

Architecture • Scope • Rules • Verification • Recovery Path

> **Document status:** LOCKED — Phase-1 architecture and scope baseline
>
> **Purpose:** If development becomes confusing, stops working, or drifts from the plan, return to this document and follow the root path. Do not invent a new architecture mid-phase.

1. Phase-1 Mission

Make the existing Work_Disk project publicly usable as a real social web application using a $0/no-credit-card target for the initial production proof. The proof is deliberately limited to two real users on two different devices. The objective is to prove real cloud persistence, cross-device synchronization, authentication, social relationships, posts, and private realtime text chat.

2. Non-Negotiable Principles

- No fake implementation: production UI must use real backend data.
- No hardcoded Render API URLs or placeholder endpoints.
- No feature is considered complete merely because the page renders.
- No paid service is introduced into Phase-1 without explicit re-evaluation.
- No assumption is treated as proof. Provider limits, payment requirements, and important behaviors must be verified from official documentation or actual controlled testing.
- Existing Work_Disk code is reused where sound; we do not recreate the product from zero unnecessarily.
- Calls are not allowed to block Phase-1; audio/video calls are Coming Soon.
- Cloud data is authoritative. Local cache is optional and never the source of truth.

3. Locked Phase-1 Architecture

- **Source control:** GitHub
- **Frontend hosting:** Cloudflare Pages
- **Authentication:** Supabase Auth
- **Primary database:** Supabase PostgreSQL
- **Realtime text chat:** Supabase Realtime
- **Initial media storage:** Supabase Storage with controlled quotas
- **Separate Render/FastAPI production server:** NOT required for Phase-1
- **Cloudflare R2:** HOLD / optional future Phase-2 component; not a Phase-1 dependency

4. Root Data Flow

GitHub → Cloudflare Pages → Work_Disk frontend → Supabase Auth / PostgreSQL / Realtime / Storage

The browser must never receive private service-role credentials. Public client configuration may be exposed as appropriate; privileged secrets remain server-side.

5. Phase-1 Feature Contract

Authentication: Signup, login, logout, session persistence, password recovery/verification where configured.

Profile: Real authenticated profile data; no fake 'No profile assembled' replacement when authoritative data exists.

Friends: Add Friend, Accept, Reject, Cancel request where implemented, Unfriend.

Blocking: Block and Unblock with database/application authorization enforcement.

Posts: Create, read/feed, delete own post.

Likes: Like/unlike with duplicate prevention and persistent counts/state.

Comments: Create/read/delete own comment with real post and author relationships.

Private chat: Private conversations and realtime text messages between authorized users.

Notifications: Basic friend, like, comment, and message notifications as required by the working UI.

Settings: Real account/privacy/notification settings saved to the database.

Media: Controlled image/small-media uploads; quota rejection rather than silent paid expansion.

6. Explicitly Out of Phase-1

- Audio calls
- Video calls
- Unlimited media/video storage
- Large-scale CDN/media infrastructure
- Guaranteed support for thousands of users
- AI recommendation systems
- Paid analytics/monitoring
- Dedicated always-on Python/FastAPI hosting
- Render deployment
- Cloudflare R2 as a required component

7. Database Root Model

- profiles
- user_settings
- friend_requests
- friends (or an equivalent normalized relationship model)
- blocks
- posts
- post_media
- comments
- likes

- conversations
- conversation_members
- messages
- notifications

Existing Work_Disk schema must be audited before migration. Do not blindly replace an existing schema with a copied snippet. Relationships, constraints, indexes, RLS policies, and existing application assumptions must be checked together.

8. Security Root Rules

- Supabase Row Level Security (RLS) is required for protected data.
- Users may modify only resources they are authorized to modify.
- Private conversation data must be readable only by conversation members.
- Blocking must be enforced by backend/database authorization, not only hidden UI buttons.
- Do not put Supabase service-role keys or other privileged secrets in frontend code.
- Client-side validation improves UX; it does not replace database authorization.

9. Storage Governor

Phase-1 does not attempt unlimited storage. Uploads are controlled by file-size and platform/user quotas. Before upload, the application checks the applicable limits. If the limit is reached, the operation is rejected with a clear message. The application must not silently upgrade, silently switch providers, or assume future billing.

Example user message: “Storage limit reached. Delete older media before uploading more.”

10. Deployment Root

- GitHub is the source-of-truth repository.
- Cloudflare Pages is the Phase-1 public frontend host.
- Production URL uses the free Pages domain initially; custom domain is not a Phase-1 requirement.
- Every production deployment must come from the repository, not from an undocumented local/manual copy.
- The old Render API architecture must not remain as a hidden runtime dependency.

11. Verification Protocol — Two Users / Two Devices

1. Create Account A on Device 1.
2. Create Account B on Device 2.
3. Login/logout/reload both accounts and verify sessions.
4. Verify each profile is real and persistent.
5. A sends friend request to B.
6. B accepts; verify friendship on both devices.
7. Test unfriend and re-add as applicable.
8. Test block/unblock and verify unauthorized interaction is actually prevented.
9. A creates a post; B sees the same post from the cloud.

10. B likes and comments; A sees the result.
11. A sends a private text message; B receives it through realtime.
12. B replies; A receives it through realtime.
13. Reload/close/reopen both devices and verify persisted state.
14. Test media upload within the controlled Phase-1 quota.
15. Verify no point points to Render or a placeholder API hostname.

12. Phase-1 PASS Criteria

- Public URL works.
- Two real accounts can authenticate.
- Data survives reload and re-login.
- Device A and Device B observe the same authoritative cloud state.
- Friend/block/unfriend flows work correctly.
- Posts/comments/likes are real and persistent.
- Private realtime text chat works.
- Protected data is enforced by RLS/authorization.
- No Render dependency remains in the production frontend.
- Phase-1 can be demonstrated without requiring a paid hosting service.

13. STOP / GO Rules

GO: A gate is documented, implemented, and tested with real data.

STOP: A provider requires payment/card contrary to the current constraint; do not workaround by assumption.

STOP: A feature depends on a missing backend service.

STOP: A UI works only with mocked/hardcoded data.

STOP: Security relies only on frontend hiding.

RECOVER: Return to Section 3 (Locked Architecture), identify the failing component, and fix only that component before expanding scope.

14. Research / Provider Status Baseline

Current working shortlist: Cloudflare Pages + Supabase + GitHub. Supabase is the Phase-1 backend foundation. Cloudflare Pages is the frontend deployment target. R2 remains unapproved as a mandatory component until its account/payment behavior is independently verified for the exact no-card requirement.

Important: free-tier limits and provider policies can change. Before a deployment decision that depends on a quota or payment rule, re-check the provider's current official documentation.

15. Existing Work_Disk Migration Rule

- Do not delete the existing Work_Disk project just because its current API path is broken.
- First map existing files/components/routes to the locked Phase-1 architecture.

- Replace broken Render API calls with the approved Supabase-backed path.
- Preserve working UI and business logic where compatible.
- Remove dead/fake/fallback runtime paths only after confirming they are not needed.
- After each migration unit, test the affected feature before moving to the next unit.

16. Root Recovery Procedure

If we get lost:

1. Stop coding new features.
2. Read this document from Sections 1–6.
3. Check which Phase-1 gate currently fails.
4. Inspect the existing repository only for that gate and its dependencies.
5. Fix the smallest real root cause.
6. Run the corresponding two-device test.
7. Update implementation notes, but do not change the locked architecture without a deliberate architecture review.

17. Phase Roadmap

PHASE 1 — Core social proof: authentication, profiles, friends, block/unblock, posts, comments, likes, private realtime text chat, controlled media.

PHASE 2 — Scale/experience: larger media strategy, stronger caching/PWA, storage expansion, moderation, performance, more users.

PHASE 3 — Communication expansion: audio/video calls and additional realtime capabilities.

Paid infrastructure is considered only when real usage proves that the free architecture is no longer sufficient.

18. Final Root Statement

WORK_DISK Phase-1 is successful only when the social platform works for two real users on two real devices with real cloud data, real authentication, real social relationships, persistent posts, and realtime private text chat under the $0/no-card target. We do not declare success from a static page, mocked data, or a green build alone.

END OF PHASE-1 ROOT DOCUMENT